

1. Основните инструменти в Visual Studio IDE

Solution Explorer

- показва **solution** (контейнер)
- вътре има:
 - **Project** (напр. Console App)
 - файлове: .cs, .csproj

Code Editor

- мястото, където пишеш C# код
- IntelliSense (подсказки)
- syntax highlighting
- грешки в реално време

Build System (MSBuild)

- не е прозорец, а **механизъм**
- чете .csproj
- знае:
 - какъв тип проект е
 - коя версия на .NET
 - как да компилира

Debugger

- breakpoints
- call stack
- locals
- threads window

NuGet Package Manager

- добавя външни библиотеки
- управлява зависимости

Компиляторът за C#

- част от **.NET SDK**
- нарича се **Roslyn Compiler**
- файл: `csc.exe`

C# код (.cs)



IL код (Intermediate Language)

Какво е IL (Intermediate Language)

- междинен език
- независим от процесор (CPU)

Резултатът е:

- `.exe` или `.dll`
- съдържа:
 - IL код
 - metadata (типове, методи)

Какво прави CLR (Common Language Runtime)

CLR е **runtime средата**, която:

Основни отговорности:

- стартира програмата
- управлява паметта (GC)
- управлява нишките
- обработва изключения
- осигурява type safety
- вика JIT

Как стартира програмата реално

1. Windows стартира процеса
2. Зарежда CLR в процеса
3. CLR намира `Main`

4. CLR създава **Main** нишката
5. Изпълнението започва

JIT компилира: IL в машинен код

Защо JIT е полезен

- оптимизира за конкретния CPU
- знае реалния runtime контекст
- може да прави по-умни оптимизации

C# код

↓ (Roslyn compiler)

IL код (.exe/.dll)

↓ (CLR)

JIT компилира методи

↓

Машинен код

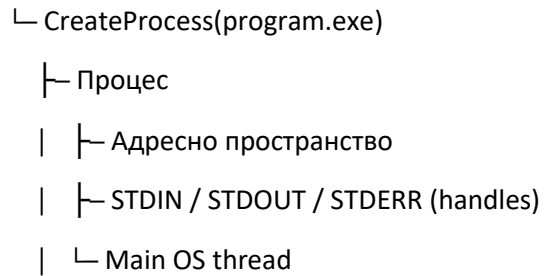
↓

Изпълнение върху CPU

СХЕМА: ЦЕЛИЯТ ПРОЦЕС (Windows + .NET)

1. Стартиране (Windows ниво)

Windows

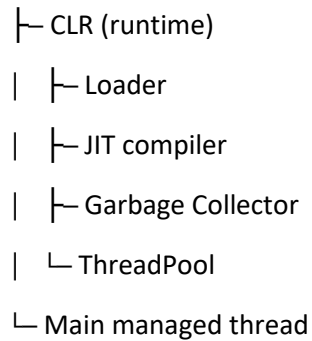


Тук още няма .NET логика.

Конзолата вече съществуват.

2. Зареждане на Runtime (CLR)

Процес



CLR се „закача“ към вече съществуващия процес.

Main нишката става managed.